

Homework 08: An Order System

Model a small order system, then close two immutability leaks you create on purpose.

Part One: Money

Create `Orders.java` in the classic form.

Write a `Money` record with `long cents` and `String currency`. Its compact constructor must:

- reject a negative amount
- reject a null currency
- **normalise** the currency by stripping whitespace and lowercasing it
- reject any currency outside a supported set of `gbp`, `eur`, `usd`

Then show that `new Money(1250, "gbp")` and `new Money(1250, "GBP ")` are `equals`, and that putting both in a `HashSet` gives one element.

One ordering detail is worth getting right: normalise before checking against the supported set, not after. Write a comment saying why.

Also add `plus(Money)` which refuses to add different currencies, and `percent(int)` which computes a percentage in whole cents.

Part Two: Two Leaks

Write a `LeakyOrder` record with a `String reference` and a `List<String> notes`, with **no** compact constructor.

Then, in `main`:

1. Build one from a list you keep a reference to
2. Add to *your* list afterwards, and print the record
3. Add to the list its accessor returns, and print the record

Both attacks will succeed. Then write `Order`, the fixed version, and show both attacks failing.

Explain It

In a comment, describe **both** leaks separately. They are not the same leak, and a fix for one does not automatically fix the other. Then say why this matters more for a record than for an ordinary class, referring back to Module 06.

Part Three: Shipping and Status

Write a `Shipping` enum with three options, each carrying a description and **its own costing rule** as constant-specific behaviour. Standard shipping is free over 50.00 and costs 3.99 otherwise. Express is always 8.99. Collection is always free.

Write an `OrderStatus` enum with four constants, each carrying an explicit **stable code** such as `"PLC"`. Add a comment saying why the code exists rather than using `ordinal()`.

Finally, write a method that gives advice per status using a `switch` expression with **no default branch**, and a comment explaining what you gain by omitting it.

Acceptance Criteria

- ☐ Runs with `java Orders.java`
- ☐ `Money` normalises currency in the compact constructor
- ☐ A comment explains the normalise-then-validate ordering
- ☐ `Money(1250, "gbp")` equals `Money(1250, "GBP ")`, shown in output
- ☐ `plus` refuses mismatched currencies
- ☐ `LeakyOrder` output shows both attacks succeeding
- ☐ `Order` output shows both attacks failing
- ☐ A comment describes the two leaks separately
- ☐ `Shipping` uses constant-specific behaviour, not a `switch`
- ☐ `OrderStatus` carries an explicit code, with a comment on why
- ☐ The advice `switch` has no `default`

Stretch

Add a `CANCELLED` handling branch to your advice method, then add a fifth constant to `OrderStatus` and **do not** update the switch. Compile it, record the exact error, and put it in a comment.

Then add a `default` branch, recompile, and describe what changed about the safety of the code.

Hint, if part two's second attack does not work

Check what your accessor returns. A record generates an accessor that hands back the field exactly as stored. If the field is a plain `ArrayList`, so is what the caller receives, and they can do whatever they like to it.